MODULE 16 · DOMAIN 6 - TERRAFORM STATE MANAGEMENT

Backends and State Locking

Where state lives, how two people are stopped from applying at once, and one pattern nearly every guide still teaches wrong.

DOMAIN 6 of 8 COURSE WEIGHT 12% MODULE 16 of 27

admin@sthithapragna.com

The backend is resolved **before Terraform evaluates anything else**. Every limitation on it follows from that.

The default backend

- **With no backend block, Terraform uses the local backend.** There is no such thing as no backend.
- It writes **terraform.tfstate** in the working directory, with a backup of the previous version beside it.
- It is a real backend with real behaviour, including **locking, using a lock file on disk**.
- What it cannot do is **be shared**. That is module four's syncing argument, and it is the reason teams move.
- It remains correct for experiments, for learning, and for anything genuinely single-user.

> ON THE EXAM
>
> Local state is still state. A question offering 'no backend' as an option is describing the local backend by another name.

The backend block

- A nested **backend** block inside the top-level terraform block, labelled with the backend type.
- Its arguments are **specific to the type**, describing where and how state is stored.
- **A configuration can provide only one backend block.** One configuration, one place state lives.
- The backend must be **available in the Terraform version you are running**. Backends are built in, not plugins.
- **Changing it requires terraform init again**, which is init's first responsibility from module five.

> ON THE EXAM
>
> Backends are built into Terraform, unlike providers. There is no registry of backends and no version argument for one.

Backend types worth recognising

BACKEND	NOTES
`local`	The default. State on disk, locked with a file
`s3`	Object storage plus native locking via use_lockfile
`azurerm`	Azure Blob Storage. Locking via blob leases
`gcs`	Google Cloud Storage. Locking supported
`kubernetes`	State in a Kubernetes secret
`http`	A generic REST endpoint you implement yourself

ON THE EXAM

You do not need to memorise every backend. You do need to know locking is per-backend, and that local is the default.

No variables in a backend block

- **A backend block cannot refer to named values.** Not input variables, not locals, not data source attributes.
- The reason is the opening slide: **the backend is resolved before those things exist**.
- **Fourth appearance of one principle** in this course: what the engine needs early cannot come from what runs late.
- The others were **plan-time keys**, **dynamic blocks and meta-arguments**, and **literal-only lifecycle settings**.
- The supported answer is **partial configuration**, which is the next slide.

> ON THE EXAM
>
> Attempting it produces a clear error saying variables may not be used here. Recognise the cause rather than the message.

Partial configuration

- **Omit some arguments from the backend block** and supply them at initialisation instead.
- **-backend-config=FILE** reads a file of settings, conventionally with an .hcl extension.
- **-backend-config="key=value"** sets one argument. Repeat it for several.
- This is how **one configuration serves several environments**, each with its own backend settings file.
- Terraform **merges** the values from all sources during init.

> ON THE EXAM
>
> An empty backend block plus a per-environment settings file is the standard pattern. The block declares the type; the file supplies the details.

Where backend credentials leak to

- **Terraform writes the backend configuration in plain text, in two places.**
- **.terraform/terraform.tfstate** holds the backend configuration for the working directory.
- **Every saved plan file captures it too**, so the plan applies against the right infrastructure.
- So HashiCorp recommends supplying credentials through **environment variables**, not the block and not -backend-config.
- Better still: leave credential arguments **completely unset** and use the target system's own conventions.

> ON THE EXAM
>
> Module five said plan files are sensitive artefacts. This is a second reason why, and it is one people rarely think about.

A saved plan carries its backend with it

- **Applying a saved plan uses the backend configuration stored in that file**, not your current settings.
- That is deliberate, and it is what stops a plan being applied against the wrong infrastructure.
- **If that configuration held time-limited credentials, they may have expired** by the time you apply.
- Another reason the two-step workflow suits automation, where plan and apply happen close together.
- Module five's staleness point again, from a different direction.

> ON THE EXAM
>
> The plan file is more self-contained than people expect. It carries configuration, values, and where the state lives.

State locking

- **Terraform locks state for any operation that could write to it.** Automatically, with no flag required.
- The purpose: stop two applies running at once and **corrupting state** or losing one person's changes.
- **Not every backend supports locking.** It is a per-backend capability, and a real selection criterion.
- If a lock is held, Terraform reports it and says **who holds it and since when**.
- Locking is not the same as access control. It prevents collisions, not unauthorised changes.

> ON THE EXAM
>
> Locking happens by default where supported. A question describing a lock being 'enabled' is usually describing choosing a backend that supports it.

The locking flags, and force-unlock

- **-lock-timeout=DURATION** waits for a held lock instead of failing immediately. Sensible in a pipeline.
- **-lock=false** disables locking entirely. Almost never the right answer.
- **terraform force-unlock** removes a lock that was left behind, for example by a crashed run.
- **It requires the lock ID**, which is deliberate friction. You cannot force-unlock by accident.
- **Never force-unlock a lock held by a running operation.** That is how you get two writers and a corrupted state.

> ON THE EXAM
>
> force-unlock is a recovery tool for an abandoned lock. If somebody's apply is still running, the lock is doing its job.

The S3 backend, and the pattern guides still teach

- **Almost every guide says: an S3 bucket for state, plus a DynamoDB table for locking.**
- **DynamoDB-based locking is deprecated**, and the dynamodb_table argument is marked as such.
- **The current mechanism is S3 native locking: use_lockfile = true.**
- It writes a **.tflock** object beside the state file in the same bucket. No second service.
- Both can be configured together during migration, so you are not forced to switch in one step.

> ON THE EXAM
>
> This is the most reliably out-of-date thing in Terraform study material. A runbook naming DynamoDB is not wrong, it is old.

The rest of a sensible S3 backend

- **bucket, key and region** are the core arguments. The key is the path to the state object.
- **Enable versioning on the bucket.** Module four called this the real protection against losing state.
- **Enable encryption**, and restrict who can read the bucket. State is sensitive data, as ever.
- **Do not put credentials in the block.** Use the platform's own credential chain or environment variables.
- Give each configuration a **distinct key**, so two configurations never share one state object.

> ON THE EXAM
>
> Versioning, encryption and access control are all still required. Native locking replaced one component, not the whole design.

The cloud block

- **A cloud block configures the HCP Terraform integration**, and replaces the older remote backend.
- **A configuration with a cloud block cannot also have a backend block.** They are alternatives.
- With HCP Terraform you **do not configure a backend at all**, because it manages state in its workspaces.
- **The cloud block does not support interpolation either**, and supports partial configuration in the same way.
- It also enables **remote operations**, where plan and apply run on HCP Terraform. Module twenty-one.

> ON THE EXAM
>
> backend and cloud are mutually exclusive. A configuration has one or the other, never both.

-migrate-state or -reconfigure?

-MIGRATE-STATE

Bring the state with you

- Reinitialise, and copy existing state across
- Terraform prompts before copying
- The normal answer when moving backends
- Your resources stay managed

-RECONFIGURE

Start fresh here

- Reinitialise, discarding existing configuration
- No state is copied at all
- For when the new backend already has the state
- Or when the old one was wrong from the start

THE TIEBREAKER

Choosing -reconfigure when you meant -migrate-state leaves the new backend empty, and Terraform will plan to create everything.

Before you migrate anything

- **HashiCorp strongly recommends manually backing up state first.**
- That advice is in the documentation. Migration touches the one unrecreatable thing.
- Terraform will **offer to migrate**, and prompts before acting.
- **-input=false suppresses that prompt**, so automation migrates silently.
- Which argues for doing backend migrations **by hand**, deliberately.

> ON THE EXAM
>
> Backing up state before migrating is documented advice, not caution for its own sake. State is the one artefact with no source.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Variables may not be used here	A backend block. Use partial configuration
One config, several environments	An empty backend block plus per-environment .hcl files
The runbook says create a DynamoDB table	Deprecated. use_lockfile on the S3 backend
Credentials appeared in a build artefact	Backend config is captured in .terraform and plan files
The new backend is empty after moving	-reconfigure was used where -migrate-state was meant
A lock is held by a crashed run	force-unlock, with the lock ID

ON THE EXAM

Most of this domain is decided by remembering that the backend is resolved first, before anything else exists.

> A platform team is standing up state storage for a new account. Their internal runbook, written three years ago, specifies an S3 bucket with versioning and encryption, plus a DynamoDB table for state locking.
>
> An engineer follows it and it works. During review, a colleague says part of it is out of date.

Which part, and what replaces it?

A. Bucket versioning is deprecated in favour of state snapshots

B. DynamoDB locking is deprecated. Use use_lockfile for native S3 locking

C. The S3 backend is deprecated entirely in favour of the cloud block

D. Nothing is out of date. The colleague is describing HCP Terraform, not the S3 backend

◆ PAUSE AND DECIDE

Which part, and what replaces it?

A Versioning deprecated

The opposite. Versioning is still recommended, and module four called it the real protection against losing state.

B DynamoDB, replaced by use_lockfile

Correct. The dynamodb_table argument is deprecated and slated for removal. Native locking writes a .tflock object beside the state.

C S3 backend deprecated

Not deprecated and widely used. HCP Terraform is an alternative, not a replacement, and choosing it is a different decision.

D Nothing is out of date

The runbook still works, which is why this survives. Working and current are different things, and the argument is marked deprecated.

THE SIGNAL

The runbook is not wrong. It is old, and it still functions, which is exactly why nobody has revisited it.

A team maintains one configuration deployed to development, staging and production, each with its own state bucket in its own account. An engineer adds a variable for the bucket name and references it from the backend block.

terraform init fails, reporting that variables may not be used here.

What is the correct approach?

A Move the backend block into a separate file so it is parsed after variables

B Use partial configuration: an empty backend block, with values supplied at init

C Define the bucket name as a local value, which backends do accept

D Maintain three copies of the configuration, one per environment

◆ PAUSE AND DECIDE

What is the correct approach?

A. A separate file

Filenames carry no meaning, as module eight established, and Terraform reads all .tf files as one configuration. Ordering is not affected.

B. Partial configuration

Correct, and it is the documented mechanism. The block names the type; a per-environment .hcl file supplies bucket, key and region at init.

C. Use a local

Locals are named values too, and the restriction covers input variables, locals and data source attributes alike.

D. Three copies

Solves it by duplicating everything, reintroducing exactly the divergence problem module fourteen described.

THE SIGNAL

The restriction is about evaluation order, not syntax, so nothing about where or how you write it will change the outcome.

A pipeline runs terraform init with -backend-config arguments that include an access key and secret for the state bucket. It then runs plan with -out and publishes the plan file for review.

A security scan flags credentials in the build artefacts. The engineer insists nothing writes credentials to disk.

What is happening?

A The scanner is matching the credentials from the pipeline log, not the artefacts

B Terraform writes backend configuration in plain text into .terraform and into every plan file

C -backend-config values are encrypted, so this must be a false positive

D The state file contains the credentials, and the plan file references state

◆ PAUSE AND DECIDE

What is happening?

A It is the log

Plausible and does not fit. The scan flagged the artefacts specifically, and the credentials genuinely are in them.

B Written to both, in plain text

Correct. HashiCorp documents this exactly, and warns that -backend-config or hardcoded values can leak credentials this way.

C They are encrypted

Nothing encrypts them. The documentation says plain text, in two separate files.

D Via the state file

State holds resource attributes, not backend credentials. The plan file captures the backend config directly, which is the real route.

THE SIGNAL

Module five said plan files are sensitive artefacts because of resource values. This is a second, separate reason.

> A team moves state from one S3 bucket to another in a different account. They update the backend block and run terraform init. Terraform asks a question they do not read carefully, and they re-run it with -reconfigure to get past the prompt.
>
> The next plan proposes to create every resource in the configuration. Nothing has been destroyed.

What happened, and what should they do?

A The resources were destroyed by the migration. Restore from the old bucket

B -reconfigure discarded the old configuration without copying state. Re-init with -migrate-state, or copy the state across

C The new bucket needs versioning enabled before Terraform can read state

D Run terraform apply. Terraform will detect the existing resources and adopt them

◆ PAUSE AND DECIDE

What happened, and what should they do?

A They were destroyed

The scenario says nothing was destroyed, and migration never destroys infrastructure. Only an apply changes anything.

B -reconfigure copied nothing

Correct. It discards the existing backend configuration and starts fresh, so the new bucket is empty and Terraform sees nothing as managed.

C Versioning first

Versioning protects against losing state. It is not a precondition for reading it, and the new bucket simply has no state in it.

D Apply will adopt them

The most dangerous option here. Terraform never adopts existing objects, as module four and module seven both established. It would create duplicates.

THE SIGNAL

The infrastructure is fine and untouched. What was lost is Terraform's knowledge of it, which is module four's point about losing state.

Module 16 recap

- **No backend block means the local backend.** Local state is still state.
- **One backend block per configuration, and no named values in it.** Use partial configuration.
- **Backend config is written in plain text** to .terraform and to every plan file. Keep credentials out.
- **Locking is automatic where the backend supports it.** force-unlock needs the lock ID, and is for abandoned locks.
- **S3 uses native locking now.** use_lockfile writes a .tflock object; dynamodb_table is deprecated.
- **-migrate-state brings state with you. -reconfigure does not.**

> ON THE EXAM
>
> cloud and backend are mutually exclusive. With HCP Terraform you configure no backend at all.